

## Advanced Concept

### \* Functional Programming :-

#### \* lambda :-

→ It is an anonymous function that is used to perform small operation.

→ lambda is a keyword.  
Syntax :-

i) `Var.name = lambda args : Expression`

ii) `Var.name = lambda args : TSB if cond else FSB : Expression`

Ex WAP to check whether no. is even or odd using lambda function.

```
even = lambda n : n%2 == 0
>> even(7)    o/p false
```

Ex WAP to print sum of two no. by using lambda.

```
s = lambda a, b : a + b
print(s(2, 3))    o/p 5
```

Ex WAP to fetch the even no. from the list.

```
l = eval(input('Enter the number:'))
```

```
even = lambda n :
```

Ex `even_add = lambda n : 'even' if n%2 == 0 else 'odd'`

Ex WAP to check whether the first char of string is vowel or not.

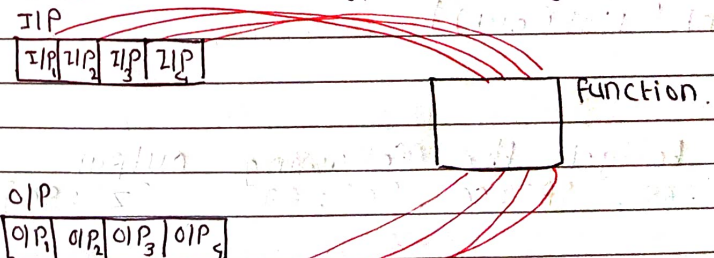
```
chk_vowel = lambda s : 'vowel' if s[0] in 'AEIOUaeiou' else 'Not vowel'
```

#### \* map() :-

→ It is a inbuilt function that is used to perform operation on each & every value that is present inside the collection.

Syntax :- `Var.name = map(function_name, collection)`  
`print(list(Var.name))`

\* Note :- Var.name does not have proper structure to display output for that reason we need type casting.



Note :- Length of I/P collection is same as length of o/p collection.



Ex Create a list to store the square values from the collection of integer 1 to 10

```
Sqr = lambda n: n**2
```

```
IP = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
out = (Sqr, IP)
```

```
print(list(out))
```

O/P [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

[OR]

```
for i in out:  
    print(i)
```

Ex WAP to store the factorial from the list & list having positive integer.

```
def fact(N):
```

```
    f = 1
```

```
    for i in range(1, N+1):
```

```
        f = f*i
```

```
    return f.
```

```
IP = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
Out = map(fact, IP)
```

```
print(list(out))
```

Ex WAP to get the following output  
{ 'A': 65, 'B': 66, 'C': 67, ..., 'Z': 90 }

```
d = lambda n: (chr(n), n)
```

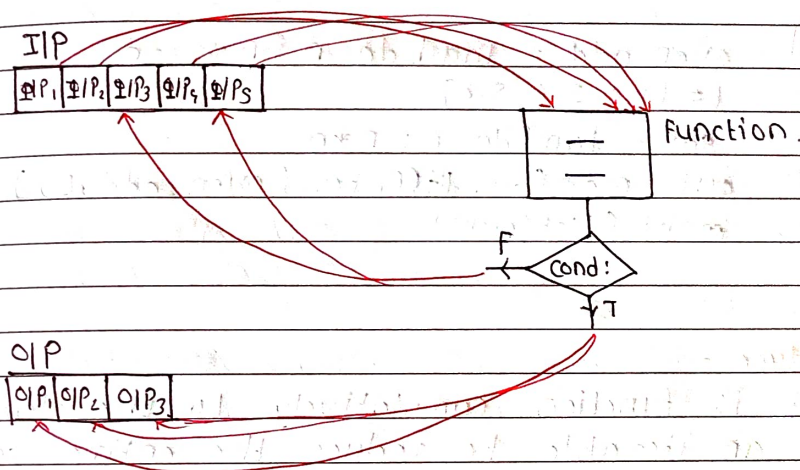
```
out = map(d, range(65, 91))
```

```
print(dict(out))
```

\* Filter() :-

→ It is a inbuilt function that is used to fetch required data from collection.

Syntax: Var Name = filter(fname, collection)  
print(list(var Name))



Note: The length output collection is same or less than input collection.

Ex WAP to fetch the even no. from the list.

```
even = lambda n: n%2 == 0
```

```
l = [1, 2, 3, 4, 5, 6, 7, 8]
```

```
out = filter(even, l)
```

```
print(list(out))
```

Ex WAP to fetch the even no. from the list & store its square.



even\_odd = lambda n : n%2 == 0  
 f = filter(even\_odd, l)  
 l = eval(input('Enter the list:'))  
 out = filter(even\_odd, l)  
 sqr = lambda m : m\*m  
 ans = map(sqr, out)  
 print(list(ans))

**OR** even\_odd = lambda n : n%2 == 0  
 l = [1, 2, 3, 4, 5, 6]  
 sqr = lambda n : n\*n  
 out = map(sqr, filter(even\_odd, l))  
 print(list(out))

### \* Reduce :-

→ It is function cumulatively to the elements of an iterable to reduce the entire sequence down to a single final value.

Syntax: from functools import reduce  
 reduce(function, iterable [, initializer])

**Ex** from functools import reduce  
 add = lambda a, b : a+b  
 l = [1, 2, 5, 6]  
 s = reduce(add, l)  
 print(s)      o/p 14

### \* Comprehension :-

→ Reducing the number of line of instruction to create collection that is known as Comprehension.

→ There are three types.

- \* List Comprehension
- \* Set Comprehension
- \* Dictionary Comprehension

#### i) List Comprehension :-

→ Creating a new list by reducing the number of lines of instructions that is known as list comprehension.

Syntax :-

var.name = [var for var in collection if cond]

**OR**

[TSB if cond else FSB for var in collection]

**OR**

[var1, var2 for var1 in collection1  
 for var2 in Collection2]

**Ex** To create a list that consist 1 to 10 integer values

out = [i for i in range(1, 11)]  
 print(out)

o/p [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

**Ex**

Create a list that contains only integer value from the given list.



Date \_\_\_\_\_  
Page \_\_\_\_\_

Ex WAP to get the following output  
OP [(A',1), ('A',2), ('A',3), ('B',1), ('B',2),  
(B',3)]

```
out = ([i,j for i in 'AB' for j in range(1,4)])  
print(out)
```

Ex    IIP [1, 2, 3, 4, 5, 6]  
        oIP [false, true, false, true, false, true]

```
out = [True if i%2==0 else False for i in range(1,7)]
print(out)
```

\* Self Comprehension :-

→ Creating the new set by reducing the number of lines of instruction that is known as Set comprehension

## Syntax :-

Var-name = { var for var in collection  
if condition }

```

    } 75B if cond else 45B for var
      in collection?

```

```
{ var1, var2 for var1 in collection1  
for var2 in collection2 }
```

Ex WAP to get following output

IP = { 6, 7, 'hi!', 'hello', 'Brandon' }  
OP = { 'hi!', 'hello', 'Brandon' }

```
print([i for i in range(10) if type(i) == str])
```

\* zip() :-

→ It is a inbuilt function that is used to perform iteration on multiple collection inside the single for loop.

## Syntax :

for var1, var2, ..., varn in zip(coll1, coll2, ..., colln):

SB

Note:- No. of variable = No. of collection

Note:- No of iteration is directly proportional to minimum length of the collection

```
for i,j in zip('AB', [6,7,12]):  
    print(i,j)
```

|     |   |   |
|-----|---|---|
| 01P | A | 6 |
|     | B | 7 |

```
Ex for i in zip('AB', [6,7,2]):  
    print(i)
```

OIP ('A', 6)  
(B', 7)



### \* Dictionary Comprehension:-

→ Creating a dictionary by reducing the number of lines of instruction that is known as dictionary comprehension.

Syntax:

Var.name = {Key:Value for Var in collection  
if condition?}

{k:v for Var1, Var2 in zip(coll1, coll2)}

{k1:v1 if condition else k2:v2 for Var1,  
Var2 in collection}

Ex WAP to get following output.  
1:1, 2:4, 3:9, 4:16

```
print({i:i**2 for i in range(1,5)})
```

Ex WAP to get following output  
IP hai hello haii

```
S = input('Enter the string:')  
print({i:len(i) for i in S if len(i) != 0})  
o/p {'haii': 4}
```

### \* Decorator:-

→ It is function that is used to give some extra information to the main task without modifying it.

→ There are two types

- \* User defined decorator
- \* Inbuilt decorator.

### \* User defined decorator:-

→ The decorator that is defined by the users as per there requirement that is known as user defined decorator.

Syntax:- Creating decorator: , address of main task/  
def deco\_name(func): function.

def inner(\*args, \*\*kwargs):

Pre task

func(\*args, \*\*kwargs)

Post task

return inner

Syntax:- Used the decorator

@deco\_name → Main task

def func(args):

50

func(values)

Ex

def insta(func):

def inner(\*args, \*\*kwargs):

print('login:')

func(\*args, \*\*kwargs)

print('logout')



return inner.

```
@insta
def AUser():
    print('Story Post')
```

```
@insta
def BUser():
    print('Like the Story')
```

```
@insta
def CUser():
    print('Comment on the Story')
```

```
AUser()    O/P    Login
              Story Post
              Logout
```

Ex Create a decorator to delay any program to run for 5 sec.

```
def delay(func):
    def inner(*args, **kwargs):
        import time
        time.sleep(5)
        func(*args, **kwargs)
    return inner
```

```
@delay
def rev_list(L):
    print(L[::-1])
```

```
>> rev_list([8,5,6])
```

Ex Create a decorator to find the total time execution to run any program.

```
def find_time(func):
    def inner(*args, **kwargs):
        import time
        st = time.time()
        func(*args, **kwargs)
        end = time.time()
        print(end-st)
    return inner
```

```
@find_time
def rev_list(L):
    print(L[::-1])
```

\* Generator :-

→ It is a function that is used to create a collection by using yield keyword.

```
Ex
def sam:
    print('hii')
    yield 1
    print('hello')
```

Sam → function address

Sam() → generator address

list(sam()) → typecasting to fetch value



• we cannot typecast in the classmate  
form of string otherwise  
return address of generator.

```
for i in sam():
    print(i)
```

O/P hii  
1  
hello

**Note:-** If the function holding return as well as yield keyword the priority is given to the yield keyword.

Ex def sam:  
 print('hiiii')  
 return 1  
 print('hellllloo')  
 yield 2

list(sam()) O/P hiiii  
[ ]

\* **Difference between yield & return keyword.**

|                | return               | yield                              |
|----------------|----------------------|------------------------------------|
| function i)    | Regular function     | i) Generator function              |
| Execution ii)  | Exits immediately    | ii) suspends & saves local states. |
| Frequency iii) | Execute exactly once | iii) can execute multiple times.   |

classmate  
Date \_\_\_\_\_  
Page \_\_\_\_\_

output

iv) A specif data type

iv) A generator object

v) Memory usage is high (Store entire dataset in RAM)

v) Memory Usage is low (computes items on demand)

Ex WAP to create a list containing the sqr of integer 1 to 10 by using generator

```
def sqr():
    for i in range(1,11):
        yield i**2
```

sqr list(sqr()) O/P [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

Ex WAP to extract the all string value from the given list by using generator.

```
def str_value(l):
    for i in l:
        if type(i) == str: # if isinstance(object, class):
            yield i        if isinstance(i, str):
```

list(str\_value(['hii', 1, 8.0, 0+4j, 'hello']))

O/P ['hii', 'hello']

Ex WAP to get following output.

O/P {'A': 65, 'B': 66, ..., 'Z': 90}

```
def get_dict():
```



```
for i in range(65, 90):
    yield chr(i)
```

```
print(dict(get_dict()))
```

Ex WAP to get following output  
 IP = ['hii', 10, 'hello', [15, 'bye', 'hii']]  
 OP = ['hii', 'hello', 'bye', 'hii']

```
def get_output(l):
    for i in l:
        if type(i) == str:
            yield i
        elif type(i) in [list, set, tuple]:
            for j in i:
                if type(j) == str:
                    yield j
```

```
list(get_output())
```

Ex WAP to get first 10 fibonacci number by using generator.

```
def fibo(s, e):
    a = 0, b = 1
    for i in range(s, e+1):
```

```
    a, b = b, a+b
```

```
    yield a
```

```
    a, b = b, a+b
```

```
list(fibo(1, 10))
```

Ex WAP to get palindrome number in given number range by using generator.

```
def get_palindrome(a, b):
```

```
    for i in range(a, b+1):
        if i == int(str(i)[::-1]):
            yield i
```

```
list(get_palindrome(1, 100))
```

[OR]

```
def get_palindrome(s, e):
    for i in range(s, e+1):
```

```
        temp = i
```

```
        rev = 0
```

```
        while i > 0:
```

```
            ld = temp % 10
```

```
            rev = rev * 10 + ld
```

```
            temp = temp // 10
```

```
        if i == rev:
```

```
            yield i
```

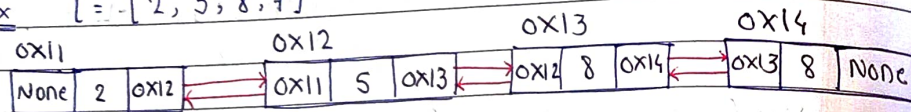
```
print(list(get_palindrome(1, 100)))
```



### \* Iterator:-

- The function that is used to perform iteration on the collection.
- There are two inbuilt functions that are used in iterators:
  - \* iter()
  - \* next()

Ex `l = [2, 5, 8, 7]`



### • iter() :-

- It is an inbuilt function that is used to fetch the address of the first value from the collection.

### • next() :-

- It is an inbuilt function that is used to fetch the current address value and point to the next address.

Ex `l = [2, 5, 8, 7]`

`i = iter(l)`

`next(i)`      O/P 2

`next(i)`      O/P 5

`next(i)`      O/P 8

`next(i)`      O/P 7

`next(i)`      exception: StopIteration

### \* File Handling :-

- Writing the data into the file or reading the data from the file.

Syntax: `Var-name = open('file name/path', 'Mode')`

### • Mode :-

i) `w` (write) :

`w+` (write & read)

`wb` (write binary)

`wb+` (write, read binary)

ii) `r` (read) :

`rt` (read, write)

`rb` (read binary)

`rb+` (read, write binary)

iii) `a` (append) :

`at` (append, read)

`ab` (append binary)

`ab+` (append, read binary)

### • Write (w) :-

- It is used to write data into the file.

i) `write()` :

- It is an inbuilt function that is used to write single line data into the file.

ii) `writeline()` :-

- It is an inbuilt function that is used to write



write multiple line of data into the file

Syntax: `Var.name.write(data)`  
`var.name.close()`

Ex `f = open('Demo.txt', 'w')`  
`f.write('Good Afternoon')`  
`f.close()`

**Note:-** To avoid the unicode error we have to used 'r' string.

Ex `f = open(r"C:\Users\.....", 'w')`  
`f.write('Hello')`  
`f.close()`

1] `with open('Demo.txt', 'w') as f:`  
`f.write('happy independence Day!')`

`with open('Demo2.txt', 'r') as f:`  
`# print(f.read())`  
`# read entire data from this file`

`with open('Demo2.txt', 'r') as f:`  
`# print(f.read())`

`# read entire data from the file`

\* **read mode(r):-**

→ It is inbuilt function that is used to read entire line data into the file  
(Demo2.txt)

1) **readline():-** Read single line from file

2) **readlines():-** Read entire data from the file & store in list

Ex `with open('file_name', 'w') as f:`  
`f.write('Hi')`

`with open('file_name', 'r') as f:`  
`print(f.read())`  
`print(f.readlines())`  
`print(f.readlines())`  
o/p ['Hello', 'Bye', 'Hello', 'Hi!']

\* **Seek():-**

→ It is inbuilt function that is used to point the cursor to the particular index position

Ex `with open('Demo.txt', 'r') as f:`  
`f.seek(0)`  
`print(f.read())`

`with open('ViratKbhti.jpg', 'rb') as vk:`  
`a = vk.read()`



```
with open('New.virat.kohli.png', 'rb') as nvk:
    nvk.write(a)
```

```
with open('redfort.png', 'rb') as rf:
    res = rf.read()
```

```
with open('flag.png', 'wb') as wf:
    wf.write(res)
```

### \* File Handling with CSV file:-

Syntax:-

```
import CSV (write mode)
with open('file.csv/path', 'w') as var_name1:
    var_name2 = CSV.writer(var_name1)
    var_name2.writerow([ ])
    OR
    var_name2.writerows([[ ]], [ ]])
```

Ex

```
import CSV
with open('first.csv', 'w', newline='') as file:
```

```
    fw = CSV.writer(file)
    fw.writerow(['Name', 'Roll', 'Age'])
    fw.writerows([[ 'xyz', 2, 21 ], [ 'Pqr', 3, 22 ]])
```

Syntax: Read Mode

```
import CSV
with open('file.csv/path', 'r') as var_name1:
```

```
var_name2 = CSV.reader(var_name1)
print(list(var_name2))
```

Ex

```
import CSV
with open('first.csv', 'r') as f:
    fr = CSV.reader(f)
    print(list(fr))
```

OR

```
for i in fr:
    print(i)
```

### \* Parsing Technique:-

- It is a way to encrypt or decrypt the data.
- There are two types of parsing technique
  - \* JSON Parsing technique
  - \* Pickle Parsing technique

#### \* JSON Parsing Technique:-

- It is a way to store the data in the form of string data

#### \* Syntax of Encryption:-

```
import JSON
Encrypt_data = JSON.dumps(data)
```

#### \* Syntax of Decryption:-

```
import JSON
decrypt_data = JSON.loads(Encrypt_data)
```



- `dumps()`:- It is inbuilt function that is used to encrypt the data.
- `loads()`:- It is inbuilt function that is used to decrypt the data.

#### Ex Encryption

```
import json
data = [10, 20, 'hii', 'hello']
encrypt_data = json.dumps(data)
print(type(data))      O/P list
print(type(encrypt_data)) O/P str
```

#### Ex Decryption

```
decrypt_data = json.load(encrypt_data)
print(type(encrypt_data)) O/P str
print(type(decrypt_data)) O/P list
```

#### • Pickle Parsing Technique:-

- Pickling is the process of converting a Python object into the byte stream so that it can be saved to a file or transmitted over a network.

#### • Syntax for Encryption

```
import pickle
encrypt_data = pickle.dumps(data)
```

#### • Syntax for Decryption

```
import pickle
```

```
decrypt_data = pickle.loads(encrypt_data)
```

#### Ex import pickle

```
data = 'Hello world'
en_data = pickle.dumps(data)
print(en_data)
# decryption
dc_data = pickle.loads(en_data)
print(dc_data)
```

#### \* Exception Handling :-

- Handling the runtime error that is known as Exception Handling.
- To make the code more smooth we are using exception handling.
- There are three way for exception handling
  - \* Specific Exception Handling
  - \* Generic Exception Handling
  - \* Default Exception Handling

#### • Specific Exception Handling:-

- When users know which type of exception will occur during the runtime then we are sp using specific exception handling.

#### Syntax:- try:

```
Program
except exception_name:
    solution
```



Ex try :

```
a = int(input('Enter the N1:'))
b = int(input('Enter the N2:'))
print(a/b)
except ZeroDivisionError:
    print("Number can't div by zero")
print('Hii')
```

O/P N1:10 N2:0  
Number can't div by zero  
Hii

• **Drawback :-** It will not handled all type of exceptions

### \* **Generic Exception Handling :-**

→ Will Handled all types of exception

Syntax:-

try :

program

→ inbuilt class

except Exception as var:

Solution

Ex try :

```
a = int(input('Enter N1:'))
b = int(input('Enter N2:'))
print(a/b)
```

```
except Exception as e:
    print(e)
```

• **Drawback :-** It will not handled KeyboardInterrupt error

### \* **Default Exception :-**

→ Default Exception will handle all the type of exception including KeyboardInterrupt exception

Syntax: try :

Program

except :

Solution

Ex try :

```
while True:
    print('Hello')
```

except :

print('Exception Handled')

→ There are Mainly 4 keywords that is used in exception handling

i) try

ii) except

iii) else

iv) finally

### • **Order of execution :-**

• try → except → finally

• try → else → finally



Ex try :  
    s = 0  
    for i in range(0,6):  
        s = s/i

except:  
    s = 5                      o/p 5

else:  
    s = 7  
finally:  
    print(s)

### \* Custom Exception :-

- Creating the user defined exception that is known as Custom Exception
- By using raise keyword we are creating User defined exception.

Ex class UserError(Exception):  
    pass

```
n = int(input('Enter the No. of product'))
if n <= 0:
    raise UserError('Product out of stock')
else:
    print('Ordered successfully')
```

### \* Assertion :-

- It is a process of creating user defined exception without using error name.

Syntax :- assert condition , 'msg'

Ex s1 = input('Enter st1')  
s2 = input('Enter st2')  
assert len(s1) == len(s2) , 'length is different'  
print(s1 + s2)

O/p @hii                      hiihey  
                hey

② hii                      length is different.  
    hello

### \* Regular Expression :-

- Write the expression to match pattern to get the required data from the huge data

Syntax :-

[A-Z]                      [1w+] → words  
[a-z]                      [1w\*] → words, space.  
[0-9] or \d  
[A-Z a-z] or \w  
[A-Z] {n}  
[A-Z] {m,n}

\ → zero or more occurrences

+ → one or more occurrences

\* → zero or more occurrences

? → zero or one occurrences



^ → start of string

\$ → end of string

· → Any single character except newline.

import re :-

i) findall() :-

→ It will return all match records from given data.

ii) finditer() :-

→ It will find the index of match data

Syntax:- import re

i) findall() :

Var.name = re.findall('Pattern', 'data')

ii) finditer() :

Var.name = re.finditer('Pattern', 'data')

Ex Expression to match phone number:

10 digits  
→ [6-9] [0-9] {9}

→ [6-9] \d {9}

import re

s = input('Enter Phone No. :')

result = re.findall('[6-9][0-9]{9}', s)

print(result)

Ex Write an expression to match data

DD/MM/YYYY [0-9]{23} | [0-9]{23} | [0-9]{43}

OR

DD/MM/YYYY \d{23} / \d{23} / \d{43}

import re

s = input()

res = re.findall('\d{23} / \d{23} / \d{43}')

print(res)

out = re.finditer('\d{23} / \d{23} / \d{43}')

print(list(out))

Ex Patter to match time.

4:15

06:30

07:45

4:03

\d{1,2} \d{1,2}

import re

s = input()

res = re.findall('\d{1,2} \d{1,2}', s)

out = re.finditer('\d{1,2} \d{1,2}', s)

print(res)

print(list(out))

Ex To match website

www.[a-z]+.[a-z]{2,3}



Ex To match USN

2AB16EC013  
 $\text{ld} [A-Z] \{2\} \text{ld} \{2\} [A-Z] \{2\} \text{ld} \{3\}$

Ex Expression to match the pin number

ABCDEF12397  
 $[A-Z] \{5\} \text{ld} \{4\} [A-Z]$

Ex To match email ID

abcd@gmail.com  
 $\text{lw}+1@ \text{lw}+1.[A-Z a-z] \{2,3\}$

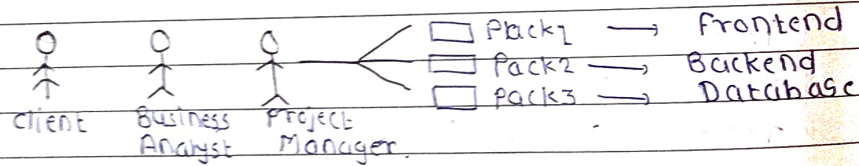
OR

abcd.efg@gmail.com  
 $\text{lw}+1.\text{lw}+1@ \text{lw}+1.[A-Z a-z] \{2,3\}$

Common Expression

$\text{lw}+1.? \text{lw} * 1@ \text{lw}+1.[A-Z a-z] \{2,3\}$

### \* Package Architecture :-



→ It is the phenomenon of dividing compute project into packages.

### • Package :-

→ It is the folder which consists of python files ('n' number of files)

### Package

i) Folder

ii) pip install package  
name or

### Module

i) File

ii) import module is compulsory

• import :- are used to bring code from modules into program.

• from :- import specific function, classes or variables from a modules.

Ex

import math

math.sqrt(64)

O/P 8.0

math.factorial(5)

O/P 120

from math import \*

sqrt(8)

2.82

pi

3.145

### • Module :-

→ Python file containing functions, classes and variables

Syntax: import module name  
module.name.var / function name()

from module name import \*  
var / fname()

from module name import var, fname()  
var / fname()



### • random module:-

→ The random module is used to generate the random number, select random item, shuffle data.

i) random() :- Generate a random float between 0.0 & 1.0

ii) randint(a,b) :- Generate a random integer between a & b

iii) choice() :- Select a random element from a sequence

iv) choices() :- Select a multiple random elements

v) shuffle() :- Randomly rearrange a list

### • Calendar Module:-

→ The calendar module is used to work with dates, months, years & calendars

i) Display a Month calendar:

- calendar.month(year, month)

ii) Display full year:

calendar.calendar(year)

iii) Check leap year:

calendar.isleap(year)

### • User defined Package:-

→ It is a package that is created by the user as per their requirement.

Package.

| File1 | File2  |
|-------|--------|
| xyz() | parc() |
| ...   | ...    |

```
import file1
file1.mname() / var
```

Ex file1

```
def valid_pw(s):
    if len(s) >= 8:
        u = 0, l = 0, sc = 0, d = 0
        for i in s:
            if 'A' <= i <= 'Z':
                u += 1
            elif 'a' <= i <= 'z':
                l += 1
            elif '0' <= i <= '9':
                d += 1
            else:
                sc += 1
        if u >= 1 and l >= 1 and d >= 1 and sc >= 1:
            return True
        return False

def valid_un(s):
    if '.' in s:
        return True
```



else :  
return false

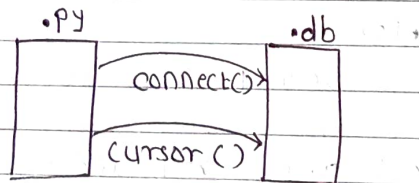
```
if __name__ == "__main__":  
    print(valid_pw("Abc@1234"))  
    print(valid_un("abc.efg"))
```

```
file2  
import file1  
un = input('Enter username :')  
pw = input('Enter password :')  
print(file1.valid_un(un))  
print(file1.valid_pw(pw))
```

### \* SQL Connection:-

#### • SQLite3 :-

→ The good thing about SQLite3 is that it is already included in python. You do not need to install a separate package.



• connect() :- It is used to establish a connection between python program and the SQLite database.

• cursor() :- After connecting to the database cursor() creates an object that allows to execute SQL queries.

#### Syntax :-

```
import sqlite3  
var1 = sqlite3.connect('data.db')  
var2 = var1.cursor()  
var1.execute('query')  
var1.commit()  
var1.close()
```

• execute() :- It is used to execute the SQL query.

• connect() :- Connect python to database.

• Syntax for creating Table :-

```
cursor.execute("CREATE TABLE TEMP  
TABLE NAME (col name 1  
'datatype' constraint ...,  
col name 2 datatype constraint  
...);")
```

• Syntax for insert the data :-

```
cursor.execute("INSERT INTO TABLE-NAME  
values (v1, v2, v3, ..., vn);")
```

• Syntax for extract the data :-

```
cursor.execute("SELECT * FROM TABLE-NAME")
```



Ex import sqlite3

```
d = sqlite3.connect('data.db')
```

```
c = d.cursor()
```

```
c.execute(""" CREATE TABLE EMP ( id number(10),  
                                     ename varchar(20), esal number(5))  
""")
```

```
c.execute(""" INSERT INTO EMP VALUES (  
            1, 'xyz', 5000)""")
```

```
c.execute(""" INSERT INTO EMP VALUES  
            (2, 'qpr', 6000)""").
```

```
d.commit()
```

```
d.close()
```

Ex fetch the data

```
import sqlite3
```

```
d = sqlite3.connect('data.db')
```

```
c = d.cursor()
```

```
result = c.execute("SELECT * FROM EMP")
```

```
print(list(result))
```

```
print(list result.fetchall()) → get all rows
```

```
print(result.fetchone()) → get one row
```

```
d.close()
```